

Guidelines de Développement Ambulon - Version Illustrée

Documentation complète avec diagrammes PlantUML de toutes les pratiques et patterns du projet Ambulon.

Table des Matières

1. [Architecture CLI - Interdiction Typer](#) 2. [Pattern argparse Obligatoire](#) 3. [Workflow Git et Branches](#) 4. [Processus de Release](#)
5. [Configuration Hiérarchique](#) 6. [Pattern de Modules de Conversion](#) 7. [Gestion des Secrets](#) 8. [Structure du Projet](#) 9.
[Workflow Complet de Développement](#)

1. Architecture CLI - Interdiction Typer

Pourquoi Typer est Banni

```

@startuml
!theme plain

title Typer vs argparse - Comparaison

rectangle "❌ TYPER (INTERDIT)" as typer #FF6B6B
rectangle "✅ ARGPARSE (OBLIGATOIRE)" as argparse #90EE90

note right of typer
  <b>❌ PROBLÈMES TYPER</b>

  • RuntimeWarning avec runpy
  • Conflits sys.argv
  • "Got unexpected extra arguments"
  • Complexité inutile
  • Dépendances externes
  • Debugging difficile

  <b>RÈGLE NON NÉGOCIABLE:</b>
  Typer a été complètement banni
  du projet. Tout code utilisant
  Typer doit être refactorisé.
end note

note right of argparse
  <b>✅ AVANTAGES ARGPARSE</b>

  • Bibliothèque standard Python
  • Stable et documentée
  • Contrôle total
  • Testable et prévisible
  • Support écosystème Python
  • Pas de dépendances

  <b>SOLUTION IMPOSÉE:</b>
  Tous les nouveaux modules CLI
  DOIVENT utiliser argparse.
end note

@enduml

```

Figure 1.1 – Comparaison CLI : Typer vs Argparse

2. Pattern argparse Obligatoire

Structure d'un Module CLI

```

@startuml
!theme plain

title Pattern CLI Obligatoire avec argparse

rectangle "commands/mon_module.py" as cmd_module #LIGHTBLUE {

}

rectangle "ArgumentParser (stdlib)" as argparser #LIGHTYELLOW {

}

rectangle "setup_logging()" as setup_log #LIGHTGREEN {

}

rectangle "load_config()" as load_cfg #LIGHTGREEN {

}

note right of cmd_module
  <b>Structure OBLIGATOIRE:</b>

  def main(argv=None):
    parser = setup_parser()
    args = parser.parse_args(argv)

    # Setup logging
    setup_logging(...)

    # Load config
    config = load_config(...)

    # Business logic
    result = fonction_metier(args, config)

    return 0 if success else 1

  if __name__ == "__main__":
    sys.exit(main())
end note

note bottom of argparser
  • add_argument()
  • add_mutually_exclusive_group()
  • parse_args(argv)
end note

note bottom of setup_log
  • level: str
  • log_file_prefix: str
end note

note bottom of load_cfg
  • config_path: Path
  • default_config: dict
end note

@enduml

```

Figure 2.1 – Pattern argparse obligatoire

Flux d'Exécution CLI

```

@startuml
!theme plain

title Flux d'Exécution Standard d'une Commande CLI

start

:Utilisateur exécute\nambul on mon-module --args;

partition "main(argv=None)" {
  :Créer ArgumentParser;

  :Configurer arguments\n(positionnal + optional);

  :parser.parse_args(argv);
  note right
    argv=None → utilise sys.argv
    argv=[...] → pour tests
  end note

  :Setup logging\n(level + file);
  note right
    - Console: INFO/DEBUG
    - Fichier: si --verbose
    - Prefix: nom module
  end note

  :Load configuration\n(YAML + ENV + CLI);
  note right
    Hiérarchie:
    CLI > YAML > ENV > defaults
  end note

  :Valider arguments;

  if (Arguments valides ?) then (oui)
    :Exécuter logique métier;

    if (Succès ?) then (oui)
      :return 0;
      stop
    else (erreur)
      :Afficher erreur;
      :return 1;
      stop
    endif
  else (non)
    :Afficher usage + erreur;
    :return 1;
    stop
  endif
}

@enduml

```

Figure 2.2 – Flux d'exécution CLI standard

3. Workflow Git et Branches

Architecture des Branches

```

@startuml
!theme plain

title Architecture des Branches Git - Ambulon

rectangle "feature/*" as feature #FFE5B4
rectangle "preprod/vX.X.X-stable" as preprod #FFFACD
rectangle "prod/vX.X.X-stable" as prod #90EE90
rectangle "main" as main #87CEEB

feature -down-> preprod : créer preprod
preprod -down-> prod : validation OK
prod -down-> main : sync main

note right of feature
  <b>Développement</b>

  <b>Actions:</b>
  • Commits conventionnels
  • cz bump
  • poetry build
  • build_offline_package.py

  <b>Durée:</b> jours/semaines
  <b>État:</b> Supprimée après merge
end note

note right of preprod
  <b>Validation</b>

  <b>Actions:</b>
  • Tests fonctionnels
  • Installation offline
  • Corrections bugs
  • Validation finale

  <b>Durée:</b> heures/jours
end note

note right of prod
  <b>Production</b>

  <b>Caractéristiques:</b>
  ☑ Immuable
  ☑ Tagguée
  ☑☑ Conservée indéfiniment
  ☑ Source de vérité

  <b>État:</b> Permanente, jamais modifiée
end note

note right of main
  <b>Miroir Production</b>

  <b>Rôle:</b>
  ☑ Lecture seule
  ☑ Sync avec prod
  ☑☑ Branche par défaut GitHub
  ☑ Toujours stable
end note

@enduml

```

Figure 3.1 – Architecture des branches Git

Voir <doc/git-workflow.md> pour les diagrammes détaillés du workflow Git.

4. Processus de Release

Workflow SemVer et Commitizen

```
@startuml
!theme plain

title Processus de Release avec SemVer

start

:Développer fonctionnalité\nsur feature branch;

:Commits conventionnels\navec cz commit;

partition "Semantic Versioning" {
  if (Type de changement ?) then (BREAKING CHANGE)
    :Version MAJOR\n3.0.2 → 4.0.0;
    note right
      feat!: ...
    ou
      BREAKING CHANGE: dans footer
    end note
  elseif (Nouvelle fonctionnalité) then (feat:)
    :Version MINOR\n3.0.2 → 3.1.0;
    note right
      feat: Add new feature
    end note
  elseif (Correction bug) then (fix:)
    :Version PATCH\n3.0.2 → 3.0.3;
    note right
      fix: Correct bug
    end note
  else (docs/style/refactor)
    :Pas de bump version;
    note right
      docs:, style:, refactor:
      test:, chore:, perf:
      → Changelog seulement
    end note
  endif
}

:cz bump --changelog;
note right
  Automatique:
  - Détermine version
  - MAJ pyproject.toml
  - MAJ __init__.py
  - Génère CHANGELOG.md
  - Crée commit + tag
end note

:poetry build;

:build_offline_package.py;

:Créer preprod/vX.X.X-stable;

repeat
  :Tests & validation;

  if (Validation OK ?) then (oui)
  else (non)
```

```
:Corriger sur preprod;  
:cz bump (patch);  
:Rebuild;  
endif  
  
repeat while (Validation OK ?) is (non) not (oui)  
  
:Créer prod/vX.X.X-stable;  
:Sync main;  
:☐ Release terminée;  
stop  
  
@endum1
```

Figure 4.1 – Processus de release avec SemVer

Types de Commits et Impact

```

@startuml
!theme plain

title Impact des Types de Commits sur la Version

object feat {
  Type = "feat:"
  Impact = MINOR
  Exemple = "3.0.2 → 3.1.0"
}

object fix {
  Type = "fix:"
  Impact = PATCH
  Exemple = "3.0.2 → 3.0.3"
}

object featBreaking {
  Type = "feat!"
  Impact = MAJOR
  Exemple = "3.0.2 → 4.0.0"
}

object breakingChange {
  Type = "BREAKING CHANGE:"
  Impact = MAJOR
  Exemple = "3.0.2 → 4.0.0"
}

object docsStyleRefactor {
  Types = "docs: / style: / refactor:"
  Impact = "Aucun"
  Note = "Changelog uniquement"
}

object testChorePerf {
  Types = "test: / chore: / perf:"
  Impact = "Aucun"
  Note = "Changelog uniquement"
}

note right of feat
  Ces commits déclenchent
  un bump de version via
  cz bump
end note

note right of docsStyleRefactor
  Ces commits apparaissent
  dans le CHANGELOG mais
  ne modifient pas la version
end note

@enduml

```

Figure 4.2 – Impact des types de commits

5. Configuration Hiérarchique

Hiérarchie de Configuration (CLI > YAML > ENV > Defaults)


```

@startuml
!theme plain

title Hiérarchie de Configuration - Priority Order

rectangle "1. Arguments CLI" as cli #FF6B6B
rectangle "2. Fichier YAML" as yaml #FFA500
rectangle "3. Variables ENV" as env #FFFACD
rectangle "4. Valeurs par Défaut" as defaults #90EE90

cli -down-> yaml : surcharge
yaml -down-> env : surcharge
env -down-> defaults : surcharge

note right of cli
  <b>Priorité MAXIMALE</b>

  <b>Exemples:</b>
  --timeout 120
  --max-retries 5
  --config custom.yaml

  ☒ Surcharge tout
  ☒ Pour tests/debug
  ☒ Usage ponctuel
end note

note right of yaml
  <b>Configuration persistante</b>

  <b>Exemples:</b>
  config/piag.yaml:
    timeout: 120
    max_retries: 5

  ☒ Config par projet
  ☒ Versionnable (*.example)
  ☒ Réutilisable
end note

note right of env
  <b>Configuration système</b>

  <b>Exemples:</b>
  PIAG_RAG_TIMEOUT=120
  PIAG_RAG_MAX_RETRIES=5

  ☒ Config par machine
  ☒ CI/CD friendly
  ☒ Sécurité (secrets)
end note

note right of defaults
  <b>Priorité MINIMALE</b>

  <b>Exemples:</b>
  DEFAULT_CONFIG = {
    'timeout': 30,
    'max_retries': 3
  }

  ☒ Fallback sûr
  ☒ Documenté dans code
  ☒ Toujours disponible
end note

@enduml

```

Figure 5.1 – Hiérarchie de configuration

Flux de Résolution de Configuration

```

@startuml
!theme plain

title Flux de Résolution de Configuration

start

:Démarrage du module\nambulonmon-module --timeout 120;

:Charger DEFAULT_CONFIG\n(valeurs par défaut);
note right
    DEFAULT_CONFIG = {
        'timeout': 30,
        'max_retries': 3,
        'url': 'http://localhost'
    }
end note

if (Variable ENV\ndéfinie ?) then (oui)
    :Surcharger avec ENV\nPIAG_RAG_TIMEOUT=60;
    note right
        config['timeout'] = 60
        (surcharge default 30)
    end note
else (non)
    :Conserver default;
endif

if (Fichier YAML\nexiste ?) then (oui)
    :Charger config/piag.yaml;
    :Merger avec config;
    note right
        yaml: timeout: 90
        config['timeout'] = 90
        (surcharge ENV 60)
    end note
else (non)
    :Pas de YAML;
endif

if (Argument CLI\nfourni ?) then (oui)
    :Appliquer args.timeout;
    note right
        args.timeout = 120
        config['timeout'] = 120
        (surcharge YAML 90)
    end note
else (non)
    :Pas d'override CLI;
endif

:Configuration finale\ntimeout = 120;

:Exécuter avec config;

stop

note right
    Résolution finale:
    CLI (120) > YAML (90) > ENV (60) > DEFAULT (30)

    Résultat: timeout = 120
end note

@enduml

```

Figure 5.2 – Flux de résolution de configuration

6. Pattern de Modules de Conversion

Architecture Standard des Modules

```
@startuml
!theme plain

title Pattern des Modules de Conversion

rectangle "commands/format1_to_format2.py" as cmd #LIGHTBLUE
rectangle "core/format1_to_format2_converter.py" as core #LIGHTGREEN

cmd -down-> core : utilise

note right of cmd
    <b>CLI Layer</b>

    <b>Fonctions:</b>
    • main(argv=None)
    • setup_parser()

    <b>Responsabilités:</b>
    • Parsing arguments
    • Configuration
    • Appel du core
end note

note right of core
    <b>Business Layer</b>

    <b>Fonctions:</b>
    • convert(input, output, options)
    • _validate_input()
    • _process()
    • _write_output()

    <b>Logique métier pure:</b>
    • Indépendante du CLI
    • Testable unitairement
    • Réutilisable
end note

@enduml
```

Figure 6.1 – Pattern des modules de conversion

Naming Conventions

```

@startuml
!theme plain

title Conventions de Nommage des Modules de Conversion

object html2md
html2md : Description = "HTML vers Markdown"

object md2html
md2html : Description = "Markdown vers HTML"

object pdf2html
pdf2html : Description = "PDF vers HTML"

object img2pdf
img2pdf : Description = "Images vers PDF"

object json2md
json2md : Description = "JSON vers Markdown"

object json2jsonl
json2jsonl : Description = "JSON vers JSONL"

object code2md
code2md : Description = "Code vers Markdown"

note right of html2md
  <b>Format: &lt;source&gt;2&lt;dest&gt;</b>

  ☒ Toujours lowercase
  ☒ Pas de underscores
  ☒ Format source en premier
  ☒ "2" comme séparateur
  ☒ Format destination en dernier

  <b>Fichiers:</b>
  - commands/&lt;source&gt;2&lt;dest&gt;.py
  - core/&lt;source&gt;2&lt;dest&gt;_converter.py
end note

@enduml

```

Figure 6.2 – Conventions de nommage

Interface CLI Unifiée

```

@startuml
!theme plain

title Interface CLI Standard pour Conversions

start

:ambulon <source>2<dest>\n<input> [options];

partition "Arguments Positionnels" {
:input_file\n(obligatoire);
note right
Fichier ou dossier source
Peut être omis pour stdin
end note
}

partition "Options Communes" {
:-O, --output\n(optionnel);
note right
Fichier de sortie
Si omis: génération auto
input.ext → input.<dest>.ext
end note

:--verbose, -v\n(optionnel);
note right
Active logging DEBUG
end note
}

partition "Options Spécifiques au Module" {
:Options propres\nau type de conversion;
note right
Exemples:
--quality (pdf)
--lang (ocr)
--format (code2md)
end note
}

:Exécuter conversion;

if (Succès ?) then (oui)
:Afficher chemin de sortie;
:return 0;
stop
else (erreur)
:Afficher erreur;
:return 1;
stop
endif

@enduml

```

Figure 6.3 – Interface CLI standard

7. Gestion des Secrets

Règles de Sécurité Strictes

```

@startuml
!theme plain

title Gestion des Secrets - Workflow de Sécurité

rectangle "🚫 AUTORISÉ" as autorise #90EE90
rectangle "🚫 INTERDIT" as interdit #FF6B6B

note right of autorise
  <b>Fichiers *.example</b>
  config/piag.yaml.example:
    token: "VOTRE_TOKEN_ICI"

  <b>Variables ENV</b>
  export PIAG_TOKEN="real_token"
  export WIKISI_API_TOKEN="secret"

  <b>.gitignore</b>
  config/*.yaml
  !config/*.example
  .env
  credentials.json

  <b>Avant CHAQUE push:</b>
  git diff HEAD | grep -i "token"

  <b>Si secret détecté:</b>
  1. NE PAS PUSHER
  2. Remplacer par placeholder
  3. Amender le commit
  4. Re-vérifier
end note

note right of interdit
  <b>Secrets dans Git (INTERDIT)</b>
  🚫 config/piag.yaml (vrais tokens)
  🚫 JWT tokens dans docs
  🚫 project_id réels dans README
  🚫 API keys dans exemples

  <b>Si secret poussé:</b>
  1. Secret = COMPROMIS
  2. Le révoquer IMMÉDIATEMENT
  3. Générer nouveau secret
  4. NE PAS juste supprimer
    (reste accessible dans historique)
end note

@enduml

```

Figure 7.1 – Workflow de gestion des secrets

Checklist Pré-Push

```

@startuml
!theme plain

title Checklist de Sécurité Avant Push

start

:Prêt à pusher\nvers GitHub;

partition "Vérifications Obligatoires" {

    :git diff --staged;

    :git diff HEAD | grep -i\n"token\\|secret\\|password\\|api_key";

    repeat
        if (Secrets détectés ?) then (oui)
            :❌ STOP - Ne pas pusher;
            :Remplacer par placeholders;
            :git commit --amend --no-edit;
        else (non)
            :grep -r "token" doc/ config/;

            if (Secrets dans docs ?) then (oui)
                :❌ STOP - Nettoyer docs;
            else (non)
                :Vérifier .gitignore;

                if (config/*.yaml\nexclus ?) then (oui)
                    :✅ Sécurité OK;
                    :git push --follow-tags;
                    stop
                else (non)
                    :❌ Ajouter à .gitignore;
                endif
            endif
        endif
    repeat while (Vérifications OK ?) is (non) not (oui)
}

@enduml

```

Figure 7.2 – Checklist de sécurité pré-push

8. Structure du Projet

Arborescence Complète


```

@startuml
!theme plain

title Structure du Projet Ambulon

rectangle "ambulon/" as ambulon_root #LIGHTBLUE {

}

note right of ambulon_root
  <b>.claude/</b>
  • GUIDELINES.md
  • PROJECT.md
  • settings.json

  <b>config/</b> (exemples versionnés)
  • piag.yaml.example
  • gitlab.yaml.example
  • wikisi.yaml.example
  • README.md

  <b>dist/</b> (généré, pas versionné)
  • ambulon-3.0.2-py3-none-any.whl
  • ambulon-3.0.2.tar.gz

  <b>dist-offline/</b> (80MB, versionné)
  • ambulon-3.0.2-offline-install.zip
  • 50 wheels de dépendances

  <b>doc/</b> (documentation PlantUML)
  • git-workflow.md
  • guidelines-illustrated.md

  <b>scripts/</b>
  • build_offline_package.py

  <b>src/app/</b> (__version__ = "3.0.2")
  • cli/ (commands)
  • conversion/
  • piag/
  • wikisi/
  • __init__.py

  <b>Fichiers racine:</b>
  • pyproject.toml
  • CHANGELOG.md
  • .gitignore
  • README.md

  <b>Principes:</b>
  ☒ Séparation commands/core
  ☒ Config exemples versionnés
  ☒ Secrets dans .gitignore
  ☒ Documentation PlantUML
  ☒ Package offline sur preprod/prod
end note

@enduml

```

Figure 8.1 – Structure du projet Ambulon

Séparation Commands / Core

```

@startuml
!theme plain

title Séparation Commands / Core - Responsabilités

rectangle "commands/pdf2html.py" as cmd #LIGHTBLUE
rectangle "core/pdf2html_converter.py" as core #LIGHTGREEN

cmd -down-> core : appelle

note right of cmd
  <b>CLI Layer</b>

  <b>Fonctions:</b>
  • main(argv=None)
  • setup_parser()

  <b>Responsabilités:</b>
  • Parsing arguments (argparse)
  • Setup logging
  • Load config
  • Appel core/
  • Gestion erreurs CLI
  • Interface utilisateur
  • Exit codes
end note

note right of core
  <b>Business Layer</b>

  <b>Fonctions:</b>
  • convert(input, output, options)
  • _validate_input()
  • _process_pdf()
  • _write_html()

  <b>Responsabilités:</b>
  • Logique métier pure
  • Conversion PDF → HTML
  • Indépendant du CLI
  • Testable unitairement
  • Pure functions
  • Réutilisable
end note

@enduml

```

Figure 8.2 – Séparation Commands/Core

9. Workflow Complet de Développement

Cycle de Vie d'une Fonctionnalité

```

@startuml
!theme plain

title Cycle de Vie Complet

start

partition "Développement" {
:Créer feature branch\ngit checkout -b feature/ma-fonctionnalite;

repeat
:Développer code;
:Tests unitaires;

if (Tests OK ?) then (oui)
:cz commit;
note right
Commit conventionnel:
feat/fix/docs/...
end note
else (non)
:Corriger code;
endif

repeat while (Fonctionnalité\ncomplète ?) is (non) not (oui)
}

partition "Versioning" {
:cz bump (3.0.2 → 3.1.0);
:poetry build;
:build_offline_package.py;
}

partition "Preprod" {
:Créer preprod/v3.1.0-stable;

repeat
:Tests installation offline;
:Tests fonctionnels complets;
:Vérification documentation;

if (Validation OK ?) then (non)
:Corriger bugs;
:cz bump (patch) 3.1.0 → 3.1.1;
:Rebuild packages;
else (oui)
endif

repeat while (Validation OK ?) is (non) not (oui)
}

partition "Production" {
:Créer prod/v3.1.0-stable;
:Tag v3.1.0;
:Push GitHub;
}

partition "Main" {
:Sync main avec prod;
:☑ Feature déployée;
}

stop

@enduml

```

Figure 9.1 – Cycle de vie d'une fonctionnalité

Matrice de Décision - Quel Type de Commit ?

```
@startuml
!theme plain

title Matrice de Décision x

start

:Modification du code;

if (Type de modification ?) then (Nouvelle\nfonctionnalité)
  if (Breaking change ?) then (oui)
    :feat!: ...\nou\nBREAKING CHANGE: dans footer;
    note right: Version: 3.0.2 → 4.0.0
    stop
  else (non)
    :feat: ...;
    note right: Version: 3.0.2 → 3.1.0
    stop
  endif
elseif (Correction\nde bug) then (oui)
  :fix: ...;
  note right: Version: 3.0.2 → 3.0.3
  stop
elseif (Documentation) then (oui)
  :docs: ...;
  note right: Pas de bump version
  stop
elseif (Refactoring) then (oui)
  :refactor: ...;
  note right: Pas de bump version
  stop
elseif (Tests) then (oui)
  :test: ...;
  note right: Pas de bump version
  stop
elseif (Style/Format) then (oui)
  :style: ...;
  note right: Pas de bump version
  stop
elseif (Performance) then (oui)
  :perf: ...;
  note right: Pas de bump version
  stop
else (Maintenance)
  :chore: ...;
  note right: Pas de bump version
  stop
endif

@enduml
```

Figure 9.2 – Matrice de décision des commits

Génération des Diagrammes

Avec PlantUML

```
# Installation
sudo apt-get install plantuml # Linux
brew install plantuml        # macOS
choco install plantuml       # Windows

# Générer tous les diagrammes depuis ce fichier
plantuml doc/guidelines-illustrated.md

# Générer en SVG (meilleure qualité)
plantuml -tsvg doc/guidelines-illustrated.md

# Générer en PNG
plantuml -tpng doc/guidelines-illustrated.md
```

Avec le service en ligne

1. Copier un bloc PlantUML depuis ce fichier 2. Aller sur <https://www.plantuml.com/plantuml/uml/> 3. Coller le code 4. Télécharger le diagramme généré

Résumé des Principes Clés

```
@startmindmap
!theme plain

title Principes Clés du Projet Ambulon

* Ambulon
** CLI
*** 📄 JAMAIS Typer
*** 📄 TOUJOURS argparse
*** Pattern standard
*** Testable (argv)
** Git Workflow
*** feature → preprod → prod → main
*** main = lecture seule
*** prod = immuable
*** preprod = validation
** Configuration
*** CLI > YAML > ENV > defaults
*** Secrets dans .gitignore
*** Exemples versionnés
*** Hiérarchie stricte
** Release
*** SemVer (MAJOR.MINOR.PATCH)
*** Commits conventionnels
*** cz bump automatique
*** Package offline généré
** Sécurité
*** Aucun secret dans Git
*** Vérification pré-push
*** Placeholders dans exemples
*** Variables ENV pour secrets
** Architecture
*** Séparation commands/core
*** Modules de conversion
*** Naming conventions
*** Documentation PlantUML

@endmindmap
```

Figure 10.1 – Résumé des principes clés

Auteur: Hervé Marchal **Version:** 3.0.2 **Date:** 2026-01-31 **Licence:** MIT

Index des Diagrammes

- 1. **Architecture CLI** - Typer vs argparse - Pattern CLI obligatoire - Flux d'exécution CLI
- 2. **Workflow Git** - Architecture des branches - Cycle de vie des branches - Processus de release - Gestion des bugs - Timeline exemple
- 3. **Versioning** - SemVer workflow - Types de commits - Matrice de décision commits
- 4. **Configuration** - Hiérarchie de configuration - Flux de résolution
- 5. **Modules de Conversion** - Architecture standard - Naming conventions - Interface CLI unifiée
- 6. **Sécurité** - Règles secrets - Checklist pré-push
- 7. **Structure Projet** - Arborescence complète - Séparation commands/core

8. **Workflow Développement** - Cycle de vie feature - Matrice de décision

9. **Résumé** - Mindmap principes clés